San Juan Islands — SSA-Race domain setup

Status: build note + research · 21 Oct 2026, **status updated 27 Oct 2026** · domains `san_juan_islands_2km` / `_1km`

> *### Where it stands, 27 Oct 2026* ***Decided and built:*** *driving data is ICON global, streamed, levels 45-120,* ***12z*** *(§6 phase 0).* `fetch_icon_global.sh` *is written and dry-run (§5g) — the "I have not shipped a* `fetch_icon_global.sh` *" line in §4 records the state on 21 Oct and is superseded by §5g. Preprocessing is source-aware rather than forked (§5h), the box can run both roads in one day under* `flock` *(§5j), and the icosahedral chain is proved end to end at 3 s per weight set with* `0 missing` *(§5k).* ***App side is complete and tested.*** *Both resolutions are registered (* `openMeteo.js` *, pinned by* `sanJuanVenue.test.ts` *), the venue is in* `lib/venues.ts` *as* `san-juan` *, observations arrive keyless from NDBC (7 stations inside 25 nm), and HRRR at 3 km already covers the venue today. The §5i follow-up "the app shows one chain" is* ***done****:* `ICONRACE_STATUS_FILES = ['status.json', 'status_usa.json']` *,* `fetchIconRaceStatus(name)` *, ModelUpdates tags the second chain, and* `iconRaceStatusChains.test.ts` *pins both.* ***Grids installed 27 Oct 2026.*** *Both Zonda outputs placed and verified: 2 km R19B07* ***8,172 cells****, 1 km R19B08* ***6,168 cells****, centre 215, extpar variable set* ***identical to solent_2km's 45*** *with nothing missing either way, md5 matched against the Zonda zips. The standalone-parent uuid pattern matches the working solent pair, so ICON will not refuse the linkage. Zonda's own* `nml_gridgen` *is kept beside each grid as provenance; the full output stays at* `/data/zonda-staging/` *(65 MB).* ***SST done 27 Oct 2026*** *— OSTIA, measured, see §5b.* `pacific` *region in* `fetch_cmems_sst.sh` *, the domain mapping in* `daily_pipeline.sh` *, the region trigger in* `cron_daily.sh` *; fetch and* `prep_sst_lam` *'s own remap both exercised on the real grid.* ***First run done, 27 Oct 2026.*** *On the operational 12z cycle (26 Oct, since today's had not published),* `REGATTA_ROOT` *redirected: fetch 6325/6325 → preprocess →* ***2 km ICON finished*** *→ nest IC/BC →* ***1 km finished****, 40 output files across the* `_hl_` *,* `_sfc_` *and* `_pbl_` *streams.* `z_ifc` *came back with* ***77 levels, top 24,169.889 m*** *— against the predicted "half level 45 = 24,170 m", so the level cut landed exactly where §4b designed it. Six defects were found and fixed on the way; see §5l.* ***Streaming done, opt-in.*** *See §5m:* `GLOBAL_STREAM=1` *holds the peak at one chunk (~11 GB) instead of one cycle (37 GB at +24 h, 64 GB at +42 h).* ***Open, in order:*** `.env` *+ a second cron entry at ~14:41 UTC; then validate against HRRR and now HRDPS too (§6 phase 7 — HRDPS is 2.5 km and brackets mast height at 10/40/80/120 m, so it is the sterner test). Tidal currents are implemented for the POINT series (SalishSeaCast, see* `src/lib/salishSeaCurrents.ts` *); the map FIELD is still Channel-only.* ***Correction to phase 4 step 13:*** `z_mc` *is not in the Zonda output — checked, in neither resolution's grid, parent, latbc or extpar file. ICON builds it at runtime from the vgrid namelist and the topography, so the standing "dump* `z_mc` *and confirm" check belongs after the first run, not after the grids arrive.*

Brief: clone the Cowes/Solent pair onto 48.5627 N, 122.8946 W; same initialisation, physics, resolution and size; same integration into SSA.

Everything below that states a fact about ExtPar or DWD was **read from the source or measured against the live service**, not assumed — two of the existing configs carry comments that turn out to be wrong, and the whole point of this note is not to inherit them.

---

1. What is built

	`san_juan_islands_2km`	`san_juan_islands_1km`
ICON grid	R19B07, ~2.05 km	R19B08, ~1.02 km
Centre	48.5627 N, −122.8946 E	same
Half-width	0.90 lat × **1.15** lon	0.45 lat × **0.43** lon
Footprint	~169 × 200 km	~63 × 100 km

	`san_juan_islands_2km`	`san_juan_islands_1km`
Namelist	**byte-identical to** `solent_2km` once the name is normalised	**byte-identical to** `solent_1km`

The lon half-widths are *not* Cowes' 1.20/0.45. A degree of longitude is longer at 48.56 N than at 50.76 N, so matching the **ground footprint** — which is what "same size" means for a weather domain — needs ×cos(50.76)/cos(48.56) = 0.9555.

The namelists being byte-identical is the point: the physics, the 63/76-level SLEVE vertical grids, the timestep and the nudging zones are the ones Cowes runs, not a re-derivation of them.

The pin is not Seattle. 48.5627 N / 122.8946 W is in the San Juan Islands, ~120 km north of the city. The 2 km domain's southern edge is 47.6627 N; Seattle city is at 47.6062 N, i.e. **6 km outside it** before the relaxation zone is counted. `sanJuanVenue.test.ts` pins that so nothing later claims the venue covers the city.

---

2. The Zonda JSON — paste these

Both files are on the box, each with the issue title and body already written:

```
ssh regatta 'sudo cat /opt/regatta-
nwp/domains/san_juan_islands_2km/grid/ZONDA_ISSUE_san_juan_islands_2km.md'
ssh regatta 'sudo cat /opt/regatta-
nwp/domains/san_juan_islands_1km/grid/ZONDA_ISSUE_san_juan_islands_1km.md'
```

Each goes into a new issue at `github.com/C2SM/zonda-request` (template `data_request.md`). The 2 km request body is:

```
{
  "zonda": { "request_name": "san_juan_islands_2km", "backend_tag": "v2.1",
             "icontools_tag": "latest", "extpar_tag": "latest" },
  "globals": { "keep_basegrid_files": false, "grid_root": 19, "grid_level": 7,
               "icorotation": 0, "icopole_lat": 90, "icopole_lon": 0,
               "centre": 215, "subcentre": 0 },
  "domains": [{
    "domain_id": 1,
    "icontools": { "parent_id": 0, "region_type": 3,
                   "center_lat": 48.5627, "center_lon": -122.8946,
                   "hwidth_lat": 0.9, "hwidth_lon": 1.15,
                   "lrotate": false, "pole_lat": 90, "pole_lon": -180 },
    "extpar": { "use_array_cache": false, "iaot_type": 1, "ilu_type": 1, "ialb_type": 1,
                "isoil_type": 3, "itopo_type": 3, "it_cl_type": 1, "iera_type": 1,
                "iemiss_type": 1, "lradtopo": false, "enable_cdnc": false,
                "enable_edgar": false, "radtopo_radius": 40000, "nhori": 24,
                "enable_art": false, "l_terra_urb": false,
                "l_use_corine": false, "infill_corine": false,
                "ilookup_table_lu": 1, "tcorr_lapse_rate": 0.0065, "tcorr_offset": 0 },
    "extpar_plots": [{ "variable_name": "topography_c" }]
  }]
}
```

The 1 km request is the same with `grid_level: 8` — but **not** the old `hwidth_lat: 0.45` / `hwidth_lon: 0.43`. See §2b: the 1 km domain was re-specified on 27 Oct 2026 from four pinned edges, which makes it `center_lat 48.5765`, `center_lon -122.9506`, `hwidth_lat 0.295`, `hwidth_lon 0.492`. The request to file is

`grid/zonda_san_juan_islands_1km_PINNED.json` on the box.

One deliberate change from the Cowes recipe: `l_use_corine` **and** `infill_corine` **are** `false`**.** CORINE is European and has no data over Washington State. ExtPar's own consistency check (`mo_landuse_routines.f90`) only accepts CORINE as an overlay on GLOBCOVER, so with `ilu_type = 1` land use falls back cleanly to GLOBCOVER 2009 alone (~300 m, global). Left `true` it would at best be ignored and at worst produce empty land use — which sets **roughness**, and therefore the whole surface layer.

2b. The 1 km domain, re-specified from the pinned edges (27 Oct 2026)

The original 1 km box was the Solent nest's footprint moved west — a round `±0.45 / ±0.43` about the venue centre. It is now the racing area Wouter pinned, with the relaxation zone added **outside** it, which is the right way round: the pins are the interior that must be clean, and the buffer is the model's own working margin, not part of the usable field.

	interior edge (pinned)	+ 10 km buffer = domain
south	48.3721	48.2822
north	48.7809	48.8707
west	-123.3060	-123.4418
east	-122.5952	-122.4594

→ `center 48.5765 / −122.9506`, `hwidth_lat 0.295`, `hwidth_lon 0.492`. Interior 45.5 × 52.3 km; domain ~65 × 73 km.

Why the grid had to be rebuilt and not just the box widened. Three of the four pins were already inside the old 1 km grid (`−123.3499..−122.4400 / 48.0908..49.0336`) with room to spare: south +21.3 km, north +18.1 km, east +1.4 km. The **west** pin was not. It sat 3.2 km inside the grid edge — i.e. *in the relaxation zone*, where the solution is being blended back to the parent and is not a 1 km answer at all. Giving it its 10 km needs the grid 6.8 km further west, and no box setting can produce a cell that was never computed.

It is smaller, not bigger. The pins span much less latitude than the old round box did, so trading that for the extra width in the west comes out ahead:

	old	pinned
lat span	0.943°	0.589°
lon span	0.910°	0.983°
cells	6,168	**~4,607** (×0.75)
nest runtime, +18 h	~47 min	**~35 min**
publish ETA	16:25 UTC = 09:25 PDT	**~16:13 UTC = 09:13 PDT**

(Runtime scaled off `solent_1km`: 5,960 cells = 45 min 13 s at +18 h. ETA is measured from today's live 12z chain — ICON global complete to +24 h at 15:14:56, 2 km phase A published 15:38.)

This is exactly the trade asked for — "happy to trim some of the bottom of the domain to keep the total size the same" — and it lands 25 % *under* the old size rather than level with it.

Measured after install: 10 km of buffer is SEVEN rows, not ten

The grid was installed on 27 Oct and audited against what ICON actually does with it, and the headline number above — "+10 km buffer" — does not mean what it was written to mean. **Kilometres of clearance are not rows of grid.** On an R19B08 grid the row-to-row spacing measured along the row normal is ~1.2–1.4 km, not 1.0 km, and a box *corner* is the worst case on two axes at once. So the 10 km buffer buys **seven rows**, and the pins sit at `refin_c_ctrl` 7–9.

That matters because the relaxation zone is counted in rows. Read off `/opt/icon/src` rather than remembered:

- `mo_impl_constants_grf.f90` — `grf_bdywidth_c = 4`, so `grf_nudge_start_c = 5`
- `mo_intp_coeffs.f90`, `init_nudgecoeffs` — `nudgecoeff_c = nudge_max_coeff · exp(−(refin_c_ctrl − 5) / nudge_efold_width)` for `0 < refin_c_ctrl ≤ nudge_zone_width + 4`
- `mo_interpol_nml.f90:274` — `config_nudge_max_coeff = 5 × (namelist value)`, so the box's `nudge_max_coeff = 0.075` is **0.375** inside the model

With `nudge_zone_width = 8` the nudged band is rows **1–12**, and rows 1–4 are the boundary-interpolation zone: not nudged but *prescribed*, the parent verbatim. So the real question for any published box is which row its corners land on. `scripts/nudge_audit.py` on the box answers it for every domain at once:

```
docker run --rm -v /opt/regatta-nwp:/work -w /work icon-nwp:riviera-opt \
    python3 -I scripts/nudge_audit.py
```

domain	worst corner	coeff	% of boundary strength	% of box nudged
san_juan_islands_1km (new)	7	0.138	37 %	48 %
san_juan_islands_1km (**old round**)	**4**	0.375	**100 %, prescribed**	52 %
solent_1km	8	0.084	22 %	36 %
porto_cervo_1km	8	0.084	22 %	23 %
porto_cervo_2km	10	0.031	8 %	9 %
la_spezia_1km	11	0.019	5 %	6 %
la_ciotat_1km, st_tropez_1km	12	0.011	3 %	0–2 %
every other 2 km	0 / 14	0	0 %	0 %

Two things follow, and they point the same way.

The regrid is a large improvement, not a marginal one. The round box's corners were at row 4 — *inside the prescribed zone*, meaning those cells were the 2 km parent's answer copied in and published as 1 km output. The pinned box moves them to rows 7–9. That is the headline benefit, and it was invisible until the grid existed to measure.

It is still the most exposed of the seven 1 km nests, one row worse than the Solent and Porto Cervo, which have been shipping for months. Closing it fully would need `hwidth_lon ≈ 0.587` and `hwidth_lat ≈ 0.346` — about **6,500 cells**, *more* than the 6,168 of the round grid it replaced, so the whole ~12 min runtime gain and the 09:13 publish go back. That trade is available but has not been taken: the exposure is a property of the entire 1 km fleet, and buying it back for one venue at the cost of the schedule is the wrong way round.

Do not re-derive the buffer in kilometres. If a future venue needs its box genuinely clear of the nudged band, the rule is **~1.3 km of `hwidth` padding per cell row**, so clearing all 12 rows wants ~16 km per side at 1 km resolution, not 10. An earlier version of `sanJuanVenue.test.ts` asserted the 10 km in degrees under the name "clear of the relaxation zone"; it passed throughout, including while the old grid's corners sat at full nudging strength. That test is renamed and now says in its own comment that it measures nominal kilometres and that `nudge_audit.py` is the authority.

The 2 km parent needs no change

Measured, rather than assumed. The 2 km grid spans `−124.0936..−121.6941 / 47.6200..49.5060`. Against the new nest plus the ~21 km (10 parent cells) a 2 km parent wants between its own relaxation zone and a nest boundary:

	required	actual	spare
south	48.0936	47.6200	+52.7 km
north	49.0593	49.5060	+49.7 km

	required	actual	spare
west	-123.7269	-124.0936	+27.0 km
east	-122.1743	-121.6941	+35.4 km

So the parent already contains the new nest comfortably, and its surplus width in the west is not waste — it is upstream fetch for the prevailing westerly down the Strait of Juan de Fuca, which is precisely the inflow the nest depends on. Trimming the 2 km to the minimum that still holds its own published box (`47.823..49.303 / −123.730..−122.060`) would save ~9 min of phase A, and is **not** recommended: it buys 9 min by removing the fetch the nest is fed from, and it costs a second Zonda request plus a fresh ExtPar.

What *did* change on the 2 km is only the published box: re-centred from the venue centre onto the nest's centre so the two are concentric. Clearances to the grid edge stay 42–52 km.

Re-measured properly after install. The table above compares against the parent's *grid extents*, which is the wrong reference for the same reason the 1 km buffer was: what the nest is fed from is the part of the parent that is free-running, outside the parent's own nudged rows 1–12. Against that, from `nudge_audit.py`:

	2 km FREE interior	1 km grid (incl. its own boundary rows)	inside by
south	47.9441	48.2599	35.2 km
north	49.1820	48.8929	32.2 km
west	−123.5915	−123.4664	**9.2 km**
east	−122.1928	−122.4339	17.7 km

The west is the tight side, at 9.2 km, and it still holds. The 2 km's own published box is entirely free (worst corner `refin_c_ctrl` 14, coefficient 0), which is the normal state for every 2 km parent here. So the conclusion stands — no 2 km regrid — but it now rests on the right measurement.

The published boxes

Both are in `src/components/weather/openMeteo.js`. The 1 km box is now **rectangular** — `halfLat 0.2044 / halfLon 0.3554`, i.e. exactly the four pins — because the domain is 45 km by 52 km and a single square `half` cannot express that: at 0.3554 it would push the north and south edges outside the computed grid, and at 0.2044 it would cut Haro Strait off the west end. `venueHalf(ven)` and `inVenueBox(ven, lat, lon)` read `halfLat` / `halfLon` with a fallback to `half`, so every other venue is untouched.

Every cell the 1 km box publishes is ~11.8 km clear of the grid edge, by construction — the box *is* the pinned interior. That is **not** the same as being clear of ICON's relaxation zone; see "10 km of buffer is SEVEN rows" above for what the audit actually measured.

As built (the installed grid, not the request): 4,660 cells, extent `−123.4664..−122.4339 / 48.2599..48.8929`, `uuidOfHGrid 6a6a14b4-1f6f-5e06-fdd6-f596de2828c0` matching the ExtPar beside it, `max(refin_c_ctrl) = 14` — which clears the `grf_bdywidth_c(4) + nudge_zone_width(8) = 12` that `init_nudgecoeffs` aborts below. The predicted ~4,607 was 1 % light; the icosahedral cut rounds outward.

The BOX also had to be rectangular — and that is on the box, not just in the app

Changing `openMeteo.js` alone would have achieved nothing, which is worth stating because it is invisible from the app side. The publisher (`/opt/regatta-nwp/scripts/publish_products.sh`) builds the served raster from `domains/<domain>/venues.csv`, and that file carried a single `half_width_deg` — still the old `−122.8946 / 48.5627, 0.35`. Install the new grid and the publisher would have gone on writing the OLD footprint (`−123.245..−122.545`), so the western strip the app now asks for would have had no cells in it: the exact symptom the regrid was done to remove, one layer further down.

So `venues.csv` takes an **optional 6th column**, the latitude half-width. Absent — every other domain — it equals column 4 and the box stays square, so nothing else changes. Present, column 4 is the longitude half-width:

```
# name,center_lon,center_lat,half_width_deg,res_deg[,half_lat_deg]
san_juan_islands,-122.9506,48.5765,0.3554,0.01,0.2044
```

Two things this cost finding out:

- **There are TWO boxes in that script, not one.** The wind raster at `res` and a second, coarser one at `SND_RES` (~0.08°) for the Skew-T sounding. Patching only the first would have left soundings published on the old square footprint — silently, since a sounding is one point and looks fine.
- **The cell count floored.** `int(2*half/res)+1` is exact for every square venue (0.30/0.01, 0.55/0.02 … are all integral) but not for a pinned half-width: 0.2044 came out 41 rows, ending **975 m short of the northern pin**, and 0.3554 ended 57 m short in the east. It now rounds up, inside a `1e-9` tolerance — the tolerance is the point, because `2*0.23/0.01` can land on either side of 46 in floating point and a naive `ceil` would then add a phantom column to St Tropez. Verified: all 12 existing venue rows produce byte-identical geometry, and the new row reaches both pins (73 × 42 = 3,066 points, lighter than the old 71 × 71 = 5,041).

The app needs no matching change for the shape: `windField.js` derives `nx` and `ny` independently from the published cell list (`[...new Set(cells.map(lon))]`), so a non-square grid already worked client-side. `grid_tab_to_json.py` emits a cell list with no raster header at all, so it is geometry-agnostic too.

The two definitions must stay in step — `venues.csv` on the box and the venue entry in `openMeteo.js`. A mismatch is silent in the worst way: the app requests a box the publisher never wrote, and the points simply grey out.

3. Terrain — the research, and why the answer is still MERIT

What the options actually are

From `src/mo_topo_data.f90` in `C2SM/extpar` — not from the config comments, which disagree with each other:

`itopo_type`	Constant	Dataset	Resolution	Coverage
1	`topo_gl`	GLOBE	~1 km	global
2	`topo_aster`	ASTER GDEM	~30 m	83 N–83 S
3	`topo_merit`	MERIT	~90 m	90 N–60 S

There is **no Copernicus GLO-30 option in ExtPar.** `domains/riviera_2km/extpar/extpar_riviera_2km.config` comments `itopo_type = 3` as "Copernicus / GLO-30 class DEM"; the Solent config comments the same value as "MERIT/REMA (~90 m)". The Solent one is right. Worth correcting in the Riviera config so it stops misleading the next reader — flagged, not changed, since it does not alter behaviour.

Why not ASTER, given "high-res terrain is important"

Because at these grid spacings the DEM is not the limiting factor:

- A **1 km** ICON cell averages roughly **120 MERIT pixels**; a 2 km cell about 500. MERIT at 90 m already oversamples the target grid by two orders of magnitude. Going to 30 m multiplies the sample count, not the information the model can hold.
- **ASTER is nominally finer but measurably dirtier.** Validation work on GDEM reports residual cloud patterns, stripe artefacts and lake anomalies, with vertical accuracy around 15 m against ~6 m for SRTM-class data; GDEM2 reduced voids "at the cost of some added noise". ASTER is also a *surface* model — canopy included. MERIT exists specifically because those errors were removed: speckle, stripes, absolute bias and tree-height bias.
- That noise is not harmless here. The DEM feeds the **sub-grid orography statistics** the SSO drag scheme uses, and the SSA 2 km domains run SSO **on**. Noise inflates the sub-grid standard deviation and so the drag — a fake terrain signal, not a sharper one.

So: `itopo_type = 3` **(MERIT) on both, matching Cowes.** ASTER is the right lever for a 500 m nest, which is already this project's written convention ("ASTER 30 m reserved for 500 m nests").

What *does* limit terrain fidelity here, and it is worth knowing

The San Juans are the harder case Cowes is not. Mount Constitution on Orcas is **734 m** against the Isle of Wight's 241 m, and the islands are small and fragmented — Shaw is ~20 km². At 1 km spacing a whole island is a handful of cells, and the differential heating of those islands is precisely what drives the local sea breeze. The constraint is **grid spacing and the land–sea mask**, not the DEM. If the forecast turns out to miss the island-scale breeze, the fix is a **500 m nest with ASTER**, not a finer DEM at 1 km.

One further lever, flagged rather than taken: `lradtopo` **is** `false` in the Cowes recipe and so here. It switches on topographic shading (horizon/skyview; `nhori = 24` and `radtopo_radius = 40000` are already set). On steep islands shading changes the differential heating that drives the breeze, so it is a real candidate — but it changes physics relative to Cowes, so it should be an A/B, not a silent default.

---

4. Initialisation — the blocker, measured

Cowes' initialisation cannot be reused. The box is driven by ICON-EU:

```
ICON-EU grid (read from the live GRIB): lon −23.50 .. 62.50,  lat 29.50 .. 70.50
San Juan Islands:                       lon −122.89           → 99.4° west of the western edge
```

The obvious substitute is **ICON global** from the same DWD feed: same model family, same `init_mode = 7` path, and EU-sourced, which matters because this project has an explicit sovereignty policy (README: "*Data: DWD, Copernicus, ISRIC, ESA — all European*", and "*not AWS/Azure/GCP even in EU regions*"). That policy rules out GFS on its own terms.

But global ICON does not fit this box, and the numbers are not close:

	ICON-EU	ICON global	ratio
One model-level file (U, step 000)	914,453 B	**3,537,302 B**	**3.87×**
Model levels	60	**120**	2×
Published grid	regular lat-lon	**icosahedral only** (0 regular-lat-lon files in the global `u/` directory)	—
Daily fetch	20 GB, 26,693 files	**~150 GB**	~7.7×

Against **50 GB free on** `/data` and a fetch that already takes 10 minutes at 16 workers. Roughly 3× over on disk, and ~77 minutes of fetch would eat the morning window on its own. Icosahedral also means `preprocess_cdo.sh` needs the 937 MB R03B07 grid description plus remap weights — a new preprocessing path, not a parameter change.

Verified 21 Oct: ECMWF open data is NOT usable here

Checked rather than assumed, and it fails on one specific thing: **ECMWF open data publishes no model levels.** It carries 14 pressure levels — 1000, 925, 850, 700, 600, 500, 400, 300, 250, 200, 150, 100, 50, 10 hPa — plus surface/single-level and 4 soil layers. Between the surface and 850 hPa there are exactly **two** levels. For a venue whose entire forecast problem is the lowest 100 m, that cannot initialise a convection-permitting LAM. (0.25°, with 0.1° added 20 Oct 2026; CC-BY-4.0; GRIB2; rolling 12 runs.) It is excellent for POINT forecasts — which is precisely how Open-Meteo already uses it in SSA — just not as driving data.

Verified 21 Oct: HRRR is the better US answer, and it reframes the design

	value
Resolution	**3 km, convection-permitting, hourly data assimilation**

	value
Covers the venue?	**Yes** — Open-Meteo's `gfs_hrrr` returns 24/24 non-null hours at 48.5627/−122.8946, elevation 0.0 (a water cell)
Vertical	**native hybrid levels** (`wrfnat`), i.e. the full PBL structure ECMWF open data lacks
Volume	651 MB/hour; +18 h ≈ **12 GB**, which fits the 50 GB free — and `.idx` files allow byte-range subsetting to cut it much further
Cost	free, no key, on AWS NODD

The consequence is that the 2 km domain may not be needed at all. In the Med the 2 km parent exists because the best public model is 7–13 km. In the US it is not:

```
Med:  ICON-EU 7 km  ->  our 2 km  ->  our 1 km
US:   HRRR 3 km (free, hourly, radar-assimilating)  ------>  our 1 km
```

That removes the 150 GB problem, halves the compute, and starts from better data than we could produce ourselves at 2 km from a 13 km global model.

Two honest caveats. HRRR is NOAA — US-sourced, against the README's EU-sovereignty policy. For a US venue that is a judgement call rather than a breach, but it is the user's call, not mine. And NOAA is mid-transition to **RRFS**: the dates in circulation disagree (31 Aug vs early Oct 2026) and NOAA says HRRR "will remain in operations" while one source says RRFS eventually replaces it with no date. RRFS is also 3 km over North America, so the successor is equivalent — but the fetch would need re-pointing.

The engineering risk in option B is real: ICON-LAM is normally initialised from ICON or IFS. HRRR is WRF-ARW on a Lambert grid with hybrid sigma levels, and getting that into `init_mode = 7` is not a parameter change — it is a new preprocessing path. The alternative, running WRF for this venue because HRRR *is* WRF and the nesting is native, means a second model stack to maintain, which cuts against everything else SSA has built.

Verified 21 Oct: ICON global IS affordable, if we stop retaining it

The 150 GB figure assumed we take everything and keep it. Checked against the live server, neither has to be true. There is no server-side subsetting (no filter service, and zero regular-lat-lon files anywhere in the global tree — both checked), but DWD already splits every field by **variable x level x step**, and that split *is* the subsetting API:

Lever	Saving	Why it is safe
Steps	**~56%**	The nest runs +18 h (that is what `solent_1km` does), not +42 h. 19 hourly BC steps instead of 43.
Levels	~30%	The LAM top is `top_height = 21000`. ICON global's top is 75 km, so every level above 21 km is fetched and discarded. The exact cut is readable from the HHL files.
Variables	~20%, optional	`qr` / `qs` could be dropped at the cost of a little spin-up. Riskiest of the three; take it last.

But the real unlock is not a lever, it is the order of operations. The pipeline currently fetches everything, *then* preprocesses. Stream it instead — per file: download, decompress, remap onto the LAM box, append, delete:

```
for each (var, level, step):  GET -> bunzip2 -> cdo remap -> append -> rm
```

Peak disk becomes a handful of 3.5 MB files plus the remapped output, and the remapped output for a 170 x 200 km box at 13 km is trivially small. **Disk stops being the constraint at all** — the 50 GB free becomes irrelevant, and the only question left is transfer TIME.

Which is measurable, and already measured: this box pulls **ICON-EU's 20 GB in 10 minutes**, i.e. ~2 GB/min. So:

	volume	transferred in
Naive (everything, +42 h)	~150 GB	~75 min

	volume	transferred in
+18 h only	**~66 GB**	**~33 min**
+18 h and levels trimmed	~46 GB	~23 min

A 03:15 start finishing the fetch by ~03:50 is inside the existing morning window. One-off cost: the remap weights, computed once from the 937 MB R03B07 grid description and reused thereafter.

So option A is no longer "needs more disk". It is "restructure the fetch to stream, and trim to the lead we actually run" — which is work, but ordinary work, on the chain we already have.

The one thing that cannot be recovered is the SPATIAL waste: each GRIB message is a single global field, so we download the planet to use a 170 x 200 km box. HRRR avoids this with `.idx` byte ranges; DWD publishes none, and the `.bz2` is whole-file compressed so HTTP ranges do not help either. That is the price of staying on ICON, and at ~33 min it is affordable.

So, three options

	Driving data	Volume	Keeps the ICON chain?	Risk
A	ICON global, +18 h, **streamed**	~66 GB transferred, **<1 GB on disk**	yes	icosahedral remap work; ~33 min fetch
B	HRRR → our 1 km	~12 GB	yes, but a NEW init path	HRRR→ICON initialisation is non-standard
C	**None — use HRRR via Open-Meteo, which already works**	0	n/a	none

A is now genuinely viable — the disk objection is gone and the fetch fits the window. It keeps one model family, one preprocessing chain and the EU-sovereignty policy intact, which B does not.

Start with C anyway. SSA already ingests HRRR, so there are 3 km forecasts for this venue *today* with no box work at all. The only thing our own nest adds is 1 km island-scale resolution — and whether that is needed is an empirical question that C answers for free. Build the chain once HRRR has been shown to miss the island-scale breeze, not before.

So I have not shipped a `fetch_icon_global.sh`. *(State on 21 Oct. Superseded: option A was chosen and §5g has the written, dry-run fetcher.)* Writing a production fetcher that cannot run here would be worse than saying so. The realistic options, in the order I would consider them:

1. **ECMWF open data IFS 0.25°** — global, regular lat-lon, EU-sourced (so the sovereignty policy holds), and perhaps an order of magnitude smaller than global ICON. ICON ingests IFS through its own init path. *Needs verifying:* I could not enumerate `data.ecmwf.int` from here, so volume and field availability are unconfirmed.
2. **ICON global with a short lead** — +12/+18 h instead of +42 h cuts the volume ~2.3×, to ~65 GB. Still over, but a bigger `/data` makes it feasible, and it keeps the model family identical.
3. **More disk.** The cheapest change if ICON global is wanted as-is.

This is the decision that gates everything else: **do not request the grids until the driving data is settled**, or the grids will arrive with nothing to run on.

4b. Which ICON global run, to be ready by 09:30 local

Measured, not estimated. ICON global publication times off the live server (Last-Modified), and the compute off Cowes' own August runs — `solent_2km` / `solent_1km` ran 1-8 Aug and San Juan is the same footprint, resolution and namelist, so nothing has to be scaled.

	measured
ICON global step 000 published	**cycle + 2h41** (00z 02:41:07, 12z 14:40:34)
ICON global complete to +24 h	**cycle + 3h14** (00z 03:13:37, 12z 15:14:56)
IC/BC preprocess	**3m 27s** (Cowes, 3 days, +/-2s)
`solent_2km` +42 h, 7 ranks	**38m 10s** (38:02 / 38:14 on the other two days)
nest IC/BC	**10s**
`solent_1km` +18 h, 7 ranks	**45m 13s** (44:57 / 45:28)

Chain for San Juan is the **Sardinia split**, not a short parent run: phase A of the 2 km to `SPLITLEAD`, then the 1 km nest, then the 2 km TAIL afterwards. Porto Cervo on 12 Sept:

```
03:29:10  phase A (+22 h of 42)
04:05:58  phase A finished — nest can start
04:06:11  2 km PUBLISHED EARLY at +22 h (full outlook follows)
05:03:43  1 km nest delivered (+18 h)
```

`SPLITLEAD = 22`, and the 22 is deliberate: the comment in `daily_pipeline.sh` records that the nest needs +21 h, so 20 aborts it with "LATBC file not found" *after* the parent has already run. BC must also reach ~2 h beyond that, because ICON's async latbc prefetcher reads ahead — so **BCLEAD 24**, 25 steps, not 22.

That is the correction to my first pass: I had the parent at +21 h with no margin and BC at 22 steps, which is the configuration that was already proven to break.

The level cut, measured

ICON global half-level heights, read off the published HHL files (`grib_get -p average`):

half-level	height	
1	75,000 m	model top
30	37,159 m	
40	27,998 m	
45	**24,170 m**	**the cut**
50	20,795 m	first level *below* the 21 km LAM top
60	15,638 m	
121	236 m	surface

Spacing near the cut is **675 m per level**, so level 49 is ~21,470 m — the first above the LAM's `top_height = 21000`. Cutting there would leave under 500 m of headroom, which is not enough for the vertical remap onto the LAM column.

Cut at full level 45: keep 45..120, drop 1..44. That is **76 of 120 levels (63.3 %), a 36.7 % saving**, with **3,170 m of headroom** above the LAM top. Model-level files are ~97 % of the fetch (10 variables x 120 levels against ~19 single/soil files per step), so the saving applies to essentially the whole volume: **87.2 GB -> 55.2 GB, 43.6 min -> 27.6 min**.

At 27.6 min the fetch finishes at T+3:09, which is *before* DWD finishes publishing +24 h at T+3:14 — so the trim moves the fetch from download-bound to **publication-bound**, and no further trimming of levels buys anything at all.

```
T+2:41  step 000 lands -> streaming fetch starts (overlaps publication)
T+3:14  fetch done    (25 BC steps, levels 45-120, ~55 GB; now PUBLICATION-bound, not download)
T+3:18  IC/BC built   (+3m27)
T+3:38  2 km phase A  (+22 h = 22/42 of Cowes' 38m10 = +20m)
T+3:38  2 km published early  (+15s)
T+3:38  nest IC/BC    (+10s)
T+4:23  1 km nest     (+45m13)
T+4:24  1 km PUBLISHED  <-- the number that matters
        ... 2 km phase B (+22->42 h) runs after this, off the critical path
```

1 km ready at cycle + 4h24 (4h35 untrimmed). Converted to the venue:

Cycle	Ready UTC	PDT (UTC−7)	PST (UTC−8)
00z	04:29	21:29 *prev day*	20:29 *prev day*
06z	10:29	03:29	02:29
12z	**16:24**	**09:24**	**08:24**
18z	22:24	15:24	14:24

PDT vs PST is simply US daylight saving: **PDT = UTC−7**, in force from the second Sunday in March to the first Sunday in November; **PST = UTC−8** the rest of the year. The 1 km is ready at a fixed 16:24 UTC either way — it is the *local* clock that moves, so the summer season is the harder one by an hour. Round the County, the second weekend of November, falls just after the switch and so runs in PST.

Use the 12z. The 1 km lands **09:24 PDT / 08:24 PST**, off an analysis 4.5 h old at the briefing. Untrimmed it is 09:35 / 08:35, which is also inside the deadline — so the trim is margin, not a rescue. Further slack if ever needed: a **+15 h nest** (45m13 → 37m40) gives 09:16 PDT.

The parent cannot be shortened below +22 h without breaking the nest, as above.

Note this is the mirror image of the Med. At Porto Cervo the venue is UTC+2 and only the 00z is ever ready in time; here, being 7-8 h west means the **12z finishes during the local morning**. That is a structural advantage of a western venue, not a tuning win.

Two numbers in that chain are NOT measured and should be treated as the risk: the ~77 GB fetch volume is ICON-EU's measured 2 GB/min applied to an estimated size, and the 3m27 preprocess is a regular-lat-lon remap — icosahedral may be slower. Both inflate the same slack the PDT case lacks.

The 2-day outlook costs nothing against the deadline, because phase B is already off the critical path — and the 2 km is published early at +22 h, so there is a usable product on screen at T+3:38 (08:38 PDT / 07:38 PST) while the 1 km is still integrating.

5. Two more gaps the venue inherits

SST has no Pacific product. `fetch_cmems_sst.sh` offers `med`, `atl` (IFREMER Atlantic) and `channel` (AMM15) — all European. This project's own setup doc calls SST "*the single biggest sea-breeze lever*", and for an island archipelago whose forecast problem is thermally driven that is not a detail. A global CMEMS L4 product is needed; the README's sovereignty note explicitly prefers Mercator/IFREMER over OSTIA, so pick accordingly.

Tidal currents do not cover it either. The currents product is AMM15, English Channel only (`currentsCovered()` gates on that). The San Juans have genuinely strong tidal streams, so this is a real absence for racing there — a separate piece of work, not a configuration flag.

5b. SST — the machinery is fine, the product is the question

`preprocess_cdo.sh::prep_sst_lam` is already **source-agnostic**, and deliberately so: the header records that "*units + variable name were the two things that bit us on the Med, so both are now DETECTED, not assumed.*" It auto-detects the variable (`analysed_sst | thetao | sst | sea_surface_temperature | sosstsst | tos`), auto-detects the units (field max < 100 ⇒ degC ⇒ +273.15), takes `seltimestep,1 -sellevidx,1` so a multi-step hourly product with a depth axis works, and renames to a canonical `analysed_sst` .

So a Pacific product is mostly a dataset-id change, not new code. The one line that must survive is already flagged in the source:

```
# Land-masked source -> bilinear remap leaves MISSING at near-coast LAM sea cells;
# swap_sst's ifthenelse would then write NaN into t_g -> RRTM abort.
# THIS WAS THE KEY MED FIX -- do NOT remove.
cdo ... remapbil,"${GRIDFILE}" ... ; cdo ... setmisstonn ...
```

A coastline as convoluted as the San Juans will produce *more* near-coast missing cells than the Med, not fewer, so `setmisstonn` matters more here.

Choosing the product. Two realistic candidates, and the trade is not the obvious one:

	resolution	sensors	sovereignty
OSTIA (CMEMS)	0.05° ≈ 5.5 km	**blends IR + microwave**	UKMO — and the README explicitly prefers "*CMEMS Med L4 (Mercator Ocean, FR), **not** OSTIA (UK Met Office)*"
MUR (GHRSST)	0.01° ≈ 1 km	IR-led	NASA/JPL, US

The San Juan channels are **2–5 km wide**, so 5.5 km barely resolves them and 1 km does. But the Pacific Northwest is cloudy, and IR-derived detail only exists in clear skies — microwave sees through cloud but only at ~25 km. MUR's extra detail is therefore least available exactly when the sky is overcast, which here is often.

Before paying for resolution, test the premise. The Med needs ~1 km SST because the lever there is a *sharp coastal SST front*. The Salish Sea is strongly tidally mixed and close to isothermal year-round — the sea-breeze forcing is the large, stable land-sea contrast, not a gradient within the water. If that holds, 5.5 km is ample and the whole question evaporates. That is a cheap thing to check against buoy data and worth doing first.

MEASURED 27 Oct 2026, and the premise is half wrong — but the conclusion holds. NDBC water temperature, every station within 60 nm of the pin that reports it, 1,078 hours they share over the 45-day realtime window:

```
46088  New Dungeness      17.9 nm   mean 11.18 °C   10.20–13.00
ptww1  Port Townsend      27.6 nm   mean 12.22      11.60–13.40
ptaw1  Port Angeles       34.1 nm   mean 10.95       9.90–12.70
46267  Angeles Point      36.6 nm   mean 10.26       8.80–12.30

across-station spread at the same hour:  median 2.00 K   p90 2.50 K   max 3.40 K
station means span 1.96 K over 37 nm
```

So it is **not** isothermal: there is a persistent ~2 K gradient, cold at the western end of the Strait of Juan de Fuca and warming eastward — upwelled, tidally mixed water against Puget Sound water, which is physically what you would expect and is a real sea-breeze lever.

But it is a BROAD gradient, not a front: 2 K over 37 nm is 0.05 K per nautical mile. Across a 2-5 km San Juan channel that is about 0.1 K. A 0.05° product puts roughly twelve cells across the whole gradient, so it resolves everything that is there; MUR's 1 km would be resolving noise, and (above) its IR detail is least available under exactly

the cloud this coast has. **So the product decision is: ~5.5 km or better is ample, pick on sovereignty rather than on resolution** — which per the README means a Mercator/IFREMER global L4 rather than OSTIA. Confirm the dataset id with `copernicusmarine describe` before wiring, as §5b already demands.

Two caveats on the measurement, both worth stating. Three of the four stations are shore-mounted CO-OPS sensors rather than open-water buoys, and a sensor in a bay reads warmer than mid-channel, so part of the 2 K is siting rather than water mass. And this is one 45-day window in September-October, not the year — Friday Harbor, the one station actually *in* the islands at 4.8 nm, reports no water temperature at all, which is why the nearest usable sensor is 18 nm away.

Whichever is chosen, confirm the dataset-id with `copernicusmarine describe` before wiring it — the existing script demands exactly that discipline, because a product id and a dataset id are not the same string.

Decided 27 Oct 2026: OSTIA — because the sovereignty-preferred product has no data here

`copernicusmarine describe` gives three candidate global L4 NRT families, and the dataset ids the subset call actually needs (which are not the product ids):

product	dataset id	source
`SST_GLO_SST_L4_NRT_OBSERVATIONS_010_001`	`METOFFICE-GLO-SST-L4-NRT-OBS-SST-V2`	OSTIA, UKMO, 0.05°
`SST_GLO_PHY_L4_NRT_010_005`	`cmems_obs-sst_glo_phy-temp_nrt_P1D-m`	GMPE ensemble, UKMO
`GLOBAL_ANALYSISFORECAST_PHY_001_024`	`cmems_mod_glo_phy-thetao_anfc_0.083deg_P1D-m`	Mercator, FR, 1/12°

The README prefers Mercator/IFREMER over OSTIA. **Here that preference cannot be honoured, and the reason is not a judgement call.** Same day, same box, nearest-neighbour at four points in the race area:

```
                Haro Str    Rosario Str   Friday Hbr   San Juan Ch
OSTIA           284.50 K      284.09        283.98       283.73      (11.4 .. 10.6 °C)
Mercator 1/12°   MISSING      MISSING       MISSING        11.82 °C
```

A global ocean *model* does not resolve a 2-5 km inland channel; a satellite L4 gridded at 5.5 km does, because its land mask keeps the channels open. 762 of OSTIA's 2,184 cells in this box carry data — the rest is land, which is correct for this coastline — and its values sit right on top of the NDBC buoys (New Dungeness 11.18 °C mean, Angeles Point 10.26, Port Townsend 12.22). The only remaining European option is the ODYSSEA **global L3**, which is not gap-free and so cannot feed `prep_sst_lam`. So: OSTIA, with the deviation from the sovereignty note recorded in the script itself rather than left to be rediscovered.

And the `setmisstonn` warning above was right, with a number. Remapping the OSTIA field onto the real 2 km grid leaves **5,689 of 8,172 LAM cells missing — 70 %** — against the Med's handful. `setmisstonn` takes that to 0 missing, 282.71-285.69 K, mean 284.19 K. Without it `swap_sst` would write NaN into `t_g` across most of the domain and RRTM would abort on the first radiation step.

Wired 27 Oct 2026 in three places, each backed up `.bak-20261027-1315`: `fetch_cmems_sst.sh` (the `pacific` region), `daily_pipeline.sh` (the per-domain SST file, which the 1 km nest inherits from its parent), and `cron_daily.sh` (fetch `pacific` when the domain or `EVENT_VENUE` is San Juan). The Med path was re-run into a temp directory afterwards and is unchanged.

5c. The icosahedral grid — four concrete hazards

The good news first: **the codebase already knows how to do this.** `preprocess_nest.sh` remaps ICON (unstructured parent) onto ICON (unstructured inner) and already passes `-setgrid` with the parent grid. The ICON-global path is therefore much closer to the NEST path than to `preprocess_cdo.sh`, whose header says plainly that it

takes "*regular-lat-lon GRIB2 fields*".

1. **ICON GRIB2 carries no horizontal coordinates.** CDO's maintainers are explicit: you must attach them with `setgrid` from DWD's grid file before any interpolation operator will work. That is the 937 MB `icon_grid_0026_R03B07_G.nc`, which has to be staged on the box.
2. `remapbil` **and** `remapbic` **cannot be used on ICON grids at all** — use `remapdis`, `remapnn` or `remapycon`. The current `REMAP_METHOD="gencon"` (conservative) is safe, and the SST `remapbil` is safe because its *source* is a regular-lat-lon CMEMS product. **The trap is the comment on line 46**, `"remapbil for winds if preferred"` — an invitation to a failure that would not exist on the Med.
3. **CDO 2.2.0 is broken on DWD global ICON GRIB2** (an eccodes padding error), fixed in 2.2.1. The image currently has **2.1.1, which is safe** — so this is a landmine rather than a blocker: an innocent CDO bump to 2.2.0 breaks ICON-global ingestion *specifically*, and nothing else. **Pin the version.**
4. **"Reference to source grid not found"** on self-generated weights. DWD's own fix is `selgrid` first, then `setgrid` on a sample GRIB2, then `gennn`.

And one that is already a scar here: **weights must be cached per source→target PAIR, not per cycle.** `preprocess_nest.sh` records that riviera_2km driving two nests collided on a shared `nest_weights.nc` with "*Size of source grid and weights differ*" — "*single-nest La Spezia never hit this.*" Generating weights from a 2.9 M-cell global source is expensive enough that it must be done once and reused, which is exactly how that collision happened. DWD also publishes prebuilt weights and target descriptions for the global grid; use them rather than generating ours.

5c-bis. The first 1 km nest run — a Mediterranean SST on a Pacific grid (27 Oct 2026)

The San Juan 1 km nest had **never run**. Its first attempt died three forecast hours in with

```
mo_srtm:srtm_setcoef: Uppermost model layer too low for RRTM.
```

which is about the model top, and had nothing whatever to do with the fault. Worth keeping because every obvious reading of that message is wrong.

What it actually was. `preprocess_nest.sh` swaps a ~1 km CMEMS SST onto the nest's sea points, and its default was a hard-coded **Mediterranean** file. `run_nest.sh` passes no `-s` at all. So the Med analysis was bilinearly remapped onto a grid in the Pacific Northwest, ~9,000 km away: no source data, an all-missing field, and the `ifthenelse` / `replace` chain wrote `t_g` **= −32768 K over every cell** — the int16 fill, straight into the ground temperature.

ICON initialised on that. The surface scheme destroyed the lowest-level momentum and `MAXABS VN` went from 17.94 m/s to **exactly zero inside time step 4**; the model then ground on for 1,681 steps on a dead atmosphere until RRTM met a column it could not classify and aborted. `0.22331346+163` in the first vertical-CFL print is the same event — Fortran drops the `E` at a three-digit exponent, so it reads like a number and is actually 0.22×10^{163}.

Why every structural check came back clean, in the order they were ruled out: the namelist is **byte-identical** to `solent_1km`'s, which works; the vertical grid is the validated 10/37/75/121/173/232 m profile (`run_nest.sh --verify-vgrid` confirms it, and that short run *succeeds*); the terrain is the **gentlest** of any 1 km nest here (657 m max against st Tropez's 1529 m); the IC and every boundary file carry real wind, 100 % finite, with no collapsed layers. Re-running st Tropez's 3 Oct nest from its own preserved IC **today** gives CFL 0.075 and zero zeros in 6,000 prints, which clears the box and the binary. Only a variable-by-variable diff of the two ICs found it, and only because `t_g` was uniform: `min == max == −32768`.

The fix, in two parts, applied to both levels.

	before	after
default SST	one hard-coded Med file for every domain	per basin: `san_juan_islands_* → pacific`, `solent_* → channel`, else `med` — the same mapping `daily_pipeline.sh` already used

	before	after
wrong file	silently remapped, fill written into `t_g`	valid-coverage fraction measured; **< 1 % refuses the swap** and keeps the parent's own `t_g`

`setmisstonn` was already in both scripts, flagged "THIS WAS THE KEY MED FIX — do NOT remove", precisely because a land-masked source leaves gaps at near-coast cells and `ifthenelse` would otherwise write NaN into `t_g` **and abort RRTM**. So this failure mode was already known and already paid for once. What `setmisstonn` cannot do is rescue a remap that returned nothing valid anywhere: there is no nearest neighbour to fill from. That is the gap the coverage check closes, and it is why the guard belongs beside the existing fix rather than replacing it.

After the fix the prep reports `SST covers 0.7238 of the inner grid`, `t_g` is 283.3–284.7 K (10–11 °C, right for the Salish Sea in late October), CFL runs 0.018–0.039, `MAXABS VN` 18 → 26 m/s, and **0 of 6,000 VN prints are zero**.

The lesson worth carrying: a basin-specific default is a trap for every domain that is not in that basin, and it fails silently because a remap with no source data is not an error. Anything defaulted per-region should either be derived from the domain or refused when it does not cover it.

5d. The repeat issues — things this project has already paid for

Every one of these was a day lost somewhere. A new domain re-trips them for free unless they are checked off deliberately.

Trap	How it shows	The rule
An unknown output variable name	ICON **aborts at startup** — the whole cycle, no partial forecast	Prove names on a non-racing day (`smoketest_tke.sh` exists for this)
`SPLITLEAD` too short	"LATBC file not found" **after** the parent has already run	22, not 20 — the nest needs +21 h
Partial `.bz2` left behind	`wget -nc` exits 0 without refetching, so the retry can **never** heal it	Delete the partial on any failure
`missing_steps` is a RELATIVE test	A whole variable lagging reads as "fetch complete"	Only safe while fetching *after* publication ends
Restart file moved out of the run dir	`-C` continuation **silently cold-starts** the tail	Leave `*_restart_*` in place
Shared remap weights	"Size of source grid and weights differ"	Cache per parent→inner pair
SST units/variable assumed	degC written into `t_g` as Kelvin	Detect both; keep `setmisstonn`
Vertical CFL	Afternoon convection trips CFL > 1 at `dtime = 10 s`	`dtime ≤ 6 s`, and test on a **full-day** run that reaches afternoon convection
MPI transport	`osc/pt2pt` deadlocks ICON's prefetch RMA at the boundary switch	`--ipc=host --cap-add=SYS_PTRACE --security-opt seccomp=unconfined`, then `MPI_MCA=""`
`www/` cycle dirs	~1 GB accumulating on a 9 GB root disk	The archive sweep in `cron_daily.sh`
Grid generated, levels unchecked	A drifted `z_mc` silently changes the whole profile	Dump `z_mc` after grid generation and confirm

5e. The sea-ice / surface-variable trap — and why it is BETTER here

The Med pipeline **fakes ten surface fields as zero**, because ICON wants a complete first guess and DWD's ICON-EU opendata feed omits them:

```
ZERO_SFC_2D   = fr_seaice freshsnow h_snow w_i snow_age hsnow_max snowfrac_lc plantevap
ZERO_SFC_SOIL = w_so_ice
gz0           = a small positive constant (roughness, not zero)
#  "All are ~0 in a summer Med."
```

Two traps are already solved there and must not be re-opened: the zero fields are templated on `t_g` / `w_so` so they **inherit the IC's time axis** (a field with no time axis, e.g. `fr_land`, makes `cdo merge` fail), and `gz0` is seeded positive rather than zero.

The trap for us is that comment: "~0 in a summer Med" is not true here. The Olympics and the Cascade foothills sit in and beside the 2 km domain and hold real snow — and Round the County sails in **November**. Zeroing `h_snow`, `freshsnw`, `snoag` and `hsnow_max` over snow-covered terrain destroys the albedo and surface energy balance, which is the thermal contrast the whole sea-breeze forecast rests on. Sea ice itself is a non-issue in the Salish Sea (`fr_seaice` really is 0), but the snow fields are not.

The good news, checked against the live server: ICON global publishes 8 of the 10.

ICON name	DWD global directory	
`fr_seaice`	`fr_ice`	published — **note the name differs**
`freshsnow`	`freshsnw`	published
`h_snow`, `w_i`, `hsnow_max`, `w_so_ice`	same	published
`snow_age`	`snoag`	published — **name differs**
plus `t_ice`, `h_ice`, `t_snow`, `rho_snow`, `w_snow`		published
`snowfrac_lc`, `plantevap`, `gz0`	—	**still synthesised**

So on ICON global we **fetch the real snow and ice state instead of inventing it**, and the seasonal landmine disappears. The cost is a handful of extra single-level files per step — negligible beside 55 GB of model levels.

But two of them need a rename, and name mismatches are precisely the class of bug that already bit this project on the Med (`ICON_RENAME_PAIRS` exists for exactly this reason). Add:

```
FR_ICE:fr_seaice     SNOAG:snow_age
```

`build_chmap` only renames pairs whose source is actually present, so adding them is safe for the ICON-EU domains too — they simply never match.

5f. Fixed: the fetch completeness test is now absolute

`missing_steps()` in `daily_pipeline.sh` flagged any step holding **fewer files than the fullest step**. That is purely relative and cannot see a whole VARIABLE lagging: every step is then short by the same amount, no step is shorter than any other, and the fetch declares itself complete with a variable missing.

It never bit because cron fired long after publication had finished. **The level trim makes fetching during publication the normal case** (trimmed fetch ends ~T+3:09, DWD finishes +24 h at ~T+3:14), so it had to be fixed before the ICON-global fetch exists, not after.

Now both tests apply — short against the fullest step, **and** short of an absolute `EXPECT_PER_STEP` floor. Verified on the box's bash:

scenario	per-step counts	result
all present	619, 619, 619	complete
one step lagging	619, **300**, 619	`001`

scenario	per-step counts	result
a whole variable lagging	**559, 559, 559**	`000 001 002`
same, old relative-only test	559, 559, 559	*complete — the bug*
more than expected (global)	760, 760, 760	complete

`EXPECT_PER_STEP` defaults to the measured ICON-EU 619 and is env-overridable. **For ICON global it must be recalibrated**: 10 model variables x 76 trimmed levels = 760, plus ~15 single/soil ≈ **775**. The waiting log line now prints the observed count beside the expectation, and the deadline path says explicitly when a uniform shortfall looks like a stale expectation rather than a late DWD — so a wrong value is visible instead of silently hanging the cycle.

5g. `fetch_icon_global.sh` — written, and what the dry run caught

On the box at `scripts/fetch_icon_global.sh`. Levels 45-120, steps 0-MAXLEAD, the extended surface set, partial-`.bz2` deletion kept, and `-x CMD` for streaming. `-n` dry-runs the manifest against the live server without downloading, which is how the following were found — none of them from reading the field list, which looks identical between the two models.

Four discrepancies, each of which would have produced a silently incomplete IC:

	ICON-EU	ICON global
`qr`, `qs`	published	**directories exist and are EMPTY — 0 files**
`snoag`, `hsnow_max`	single-level	**time-invariant** (one token-less file per cycle)
`w`	with the rest	**half-level field — 77 levels where the others have 76**

`qr` / `qs` are a real gap, not a difference. A straight copy of the Med field list yields an IC with two tracers missing and nothing says so. They are precipitating hydrometeors that spin up within an hour or two, so preprocessing must **zero-fill them in 3D** — the same treatment as the surface fields. That has to be an explicit decision in the code, not an omission.

`snoag` / `hsnow_max` filed as time-invariant is the nastier one: asked for as single-level they return *nothing at all*, which is how a snow field quietly becomes absent in a domain that has real snow — exactly the trap §5e is about, arriving by a different door.

`w` being a half-level field is a 76-vs-77 error that would hold the completeness gate open for ever, since no step would ever reach the expected count.

The expectation now self-derives and agrees with reality. The script computes `EXPECT_PER_STEP = 7xNKEEP + 1x(NKEEP+1) + 13` from its own field lists and writes it to `.expect_per_step`, so the absolute gate in §5f calibrates from the CONFIG rather than from whatever the server happens to have listed mid-publication — which is the blind spot the gate exists to close. Measured against the live feed: **computed 622, observed 622**, with step 000 at 647 (the extra 25 are soil, which is analysis-time only).

Verified by a real fetch (step 000, levels 119+, streamed): 61/61 files downloaded, decompressed, handed to the hook and deleted, leaving **32 KB** on disk against tens of MB of data. Disk is no longer the constraint.

It must run INSIDE the ICON container. `bunzip2` is not on the box host — run on the host every single file fails to decompress, which is exactly what the first attempt did. `fetch_icon_eu.sh` has always run in the container for the same reason.

5h. Phase 3 done — preprocessing is source-aware, not forked

`preprocess_cdo.sh` now takes `-S eu|global` and `-G <icosahedral grid>`. **One implementation, two roads** — a fork would drift, and this script is the only place that knows how IC/BC are assembled.

	`-S eu` **(default)**	`-S global`
file prefix	`icon-eu_*`	`icon_global_*`
before every remap	nothing	`cdo setgrid,<SRCGRID>`
weights file	`remap_weights_<domain>.nc` *(name unchanged)*	`remap_weights_global_<domain>.nc`
invented surface fields	all 8 + `w_so_ice`	**only** `snowfrac_lc`, `plantevap` — the rest are real
renames added	—	`FR_ICE:fr_seaice`, `SNOAG:snow_age`, `W_SO_ICE:w_so_ice`

Two guards refuse an impossible combination **up front** rather than mid-cycle: `-S global` without `-G` (ICON GRIB2 carries no coordinates), and a `remapbil` / `remapbic` method against an ICON source (CDO cannot apply either). The second exists because the old line-46 comment actively invited it.

`qr` / `qs` need no zero-fill after all. The baseline EU IC **does not contain them either** — so ICON already runs without them and absence is tolerated; `normalise_tracer_levels` only has to fix a tracer that is present but SHORT, which is the case that aborts latbc. One less thing invented.

Proving the Med and Cowes domains are untouched

Not asserted — measured. The 3 Oct ICON-EU fetch is still on disk, so the *current* script was run first, then the patch applied, then the same command re-run:

	before	**after**	`cdo diffn`
`ic_riviera_2km`	28,308,419 B	28,308,399 B	**0 differing fields**
`latbc_…00`	27,616,386 B	27,616,386 B	**0**
`latbc_…01`	27,616,386 B	27,616,386 B	**0**

Variable set identical (31 fields). The IC's 20-byte difference is the NetCDF `history` attribute, which records the CDO command line; the data is identical.

That test earned its place immediately. The first patched run built the IC and then failed every boundary file with "*no files for step 000*". `build_bc_one` runs in `xargs` SUBSHELLS, and `SRCPREFIX` was not in the `export` list — so the glob became `_*_<stamp>_` and matched nothing. The IC (main shell) was fine, the BCs (subshells) were silently empty. Exactly the failure mode that would have reached a race morning.

5i. The parallel road — what still has to change

The two chains are **naturally separated in time**, which is the thing that makes this viable at all: the Med/Cowes 00z chain runs 03:15-05:30 UTC, and the San Juan 12z chain runs ~14:41-16:24 UTC. Nine hours apart on one 8-core box, no contention, no queueing. The venue being 7-8 h west is what buys that.

Preprocessing is done. These are the remaining single-valued assumptions:

What	**Today**	**Needs**
`daily_pipeline.sh`	`RUN="00"` hardcoded	env-overridable, so the 12z chain can be the same script
fetch	always `fetch_icon_eu.sh` into `icon-eu/`	a `DRIVER` switch to `fetch_icon_global.sh` / `icon-global/`
`EXPECT_PER_STEP`	one constant	read `.expect_per_step`, which the global fetcher already writes
ICON concurrency	nothing stops two runs	**a lock** — 7 ranks each on 8 cores if one chain overruns

What	Today	Needs
`publish_status.sh`	writes ONE `status.json`	per-chain, or merged; two chains would overwrite each other
disk hygiene	prunes `icon-eu/20*`	prune `icon-global/` too
`.env`	single `DOMAINS` / `NESTS` / `EVENT_VENUE`	one set per chain
cron	one entry at 03:15	a second at ~14:41

The lock is the one that matters: the time separation makes collision unlikely, not impossible, and two ICON runs sharing 8 cores is the kind of failure that produces a late forecast on both roads at once.

5j. Phase 3b done — the box can now run both roads in one day

The ICON lock is the important one. Two chains share 8 cores; if one overruns, 7 ranks each makes BOTH forecasts late, which is worse than one waiting. `run_icon.sh` now takes an exclusive lock around the single `launch` call.

The lock lives on `/data`, which is the only path bind-mounted into every invocation (`cron_daily`, `testrun_v2`, `smoketest` all pass `-v /data:/data`). A lock under `${ROOT}` or `/tmp` would be private to each container and protect nothing — the two chains are separate `docker run`s, not threads. Proven with two separate containers:

```
19:06:28 CHAIN-A acquired lock / RUNNING
19:06:34 CHAIN-A done
19:06:34 CHAIN-B acquired lock      <- waited, did not overlap
19:06:36 CHAIN-B done

19:06:36 LONG-A acquired lock
19:06:39 SHORT-B TIMED OUT waiting  -> exit 99, rather than oversubscribing
```

The wait is bounded (`ICON_LOCK_WAIT`, 3600 s) so a wedged run is an alert, not a hang. Without `flock`, or where `/data` is not writable, it logs that it is unserialised and behaves exactly as before.

Everything else is parameterised, with the existing defaults preserved:

	default (Med/Cowes)	San Juan
`RUN`	`00`	`12`
`DRIVER`	`eu`	`global` (+ `ICON_GLOBAL_GRID`, refused at startup if unset)
source dir	`icon-eu/`	`icon-global/`
fetcher	`fetch_icon_eu.sh`	`fetch_icon_global.sh`
preprocess args	*(none)*	`-S global -G <grid>`
`EXPECT_PER_STEP`	619	**read from** `.expect_per_step`, which the global fetcher writes
`STATUS_NAME`	`status.json`	`status_usa.json`
disk hygiene	prunes `icon-eu/`	**and** `icon-global/`

Verified: with nothing set it still resolves to `RUN=00 DRIVER=eu SRCDIR=icon-eu FETCHER=fetch_icon_eu.sh PPSRC=[]` — so preprocess receives no `-S` / `-G` and falls to its `eu` default, which §5h already proved produces identical data. `DRIVER=global` without `ICON_GLOBAL_GRID` refuses at startup; with it, it resolves to `icon-global` / `fetch_icon_global.sh` / `-S global -G …`.

Two follow-ups, neither a code change:

- **The app shows one chain.** `ModelUpdates.jsx` reads `icon-race/status.json`, so the San Juan chain publishes `status_usa.json` and is simply absent from the Admin table until the component learns to read both. Deliberate: the alternative was two chains overwriting one file and each erasing the other's cycle.
- **Cron and** `.env` are deployment, not code, and should not be added until there is a grid to run. A second entry at ~14:41 UTC with its own `DOMAINS` / `NESTS` / `EVENT_VENUE`.

5k. Phase 1 proved — the icosahedral chain works, and it is cheap

Done while waiting on the Zonda grids, which is exactly the right use of that window: this was the single biggest unknown on the ICON-global road.

Staged: `icon_grid_0026_R03B07_G.nc` at `/data/icon-global-static/` — 937 MB down in **under 8 seconds** (~117 MB/s), 1.79 GB decompressed, 43 GB still free. Verified a real ICON grid: **2,949,120 cells**, `grid_root 3 / grid_level 7`, `uuidOfHGrid` present.

No prebuilt weights exist. DWD's CDO library holds the grid file and nothing else, so we generate our own — which turns out not to matter at all (below).

The chain works end to end, proven on a real global file remapped onto `solent_2km` (the same size as San Juan, so representative):

```
setgrid  -> unstructured, points=2949120, nvertex=3
            clon -3.141593..3.141593 radian, clat -1.56928..1.56928 radian
gencon   -> 2.95M global cells -> 7,884 LAM cells in 3 SECONDS, 249 MB weights
remap    -> u: min 1.0390  mean 7.3689  max 14.853 m/s, 0 missing
```

Weight generation is 3 seconds, not "expensive". That assumption shaped the earlier plan — it was wrong, and cheaply so. There is no reason to cache weights across cycles for performance; the per-pair caching stays for CORRECTNESS (the stale-weights collision), not speed.

`0 missing` **matters too:** the conservative remap covers the whole LAM box from the global source, so there is no coastal gap to patch the way the SST path needs `setmisstonn`.

One real trap, and the existing code already avoids it. Without `-f nc`, CDO infers GRIB output from the GRIB input and then:

```
ECCODES ERROR : pack_bytes: Wrong size (20) for section2Padding
Warning (gribapiDefGridUnstructured): GRIB library doesn't support unstructured
                                      grids, grid information will be lost!
```

That is the same eccodes padding error the CDO forum attributes to 2.2.0 — it appears on **2.1.1 too**, on the GRIB *write* path. `remap_step` already passes `-f nc4`, so the data path is safe. Checked separately: `gen_weights` does **not** pass `-f nc4` and does not need to — CDO writes weights as netCDF regardless, and a remap through those weights returns bit-identical values (1.0390 / 7.3689 / 14.853). **No change required to §5h.**

So phase 1 steps 1 and 3 are complete, and step 2 (pin CDO at 2.1.1) is now better motivated: 2.1.1 demonstrably handles the global GRIB, and the padding error is a write-path hazard that `-f nc4` sidesteps rather than a version blocker.

5l. What the first run actually found

The fetch was never the problem. Six defects, all in the gate and the naming around it:

	what it was	why it mattered
deadline	`FETCH_DEADLINE_UTC="0640"` hardcoded	at 13:34 UTC it was 7 h past, so the wait loop gave up on its FIRST pass while the fetch was still running. Now RUN-relative: 00z keeps 0640/0730, 12z gets 1840/1930
step 000	counted as a peer of the others	global files its time-invariants under step 000, so 647 against 622 made 647 the bar and steps 001+ could never reach it. The gate waited, every two minutes, for a fetch that had finished
observed count	`MISSING_OBSERVED_MAX` set inside `$(...)`	a subshell, so discarded — "observed 0 files/step" **on the Med too**, which made the log's own advice ("set EXPECT_PER_STEP to it") advice to set it to zero
log strings	"ICON-EU steps", "00z incomplete"	named the wrong model and the wrong cycle on the global road
`FRESHSNW`	renamed to `freshsnw`	ICON wants `freshsnow` and aborted in `fetch_dwdfg_sfc`. §5e had flagged this exact ambiguity to verify on the first global run — it verified wrong
`HSNOW_MAX`	in no rename pair at all	would have failed one step later, the same way

The last two are the instructive pair: `FR_ICE:fr_seaice` and `SNOAG:snow_age` already mapped to ICON's INTERNAL names, and `FRESHSNW:freshsnw` was the odd one out — a faithful lowercase of the DWD shortName, which is not what ICON reads.

5m. Streaming — and why the per-file hook the fetcher already has is the wrong granularity

The box's standing disk idiom is cron_daily's prune of the previous cycle, and for ICON-EU it is enough: one cycle is ~20 GB against 43 GB free. The global road breaks that. **Measured on the rehearsal: 622 files and 1.5 GB per step**, so

```
BCLEAD=24  ->  25 steps  ->  ~37 GB     fits alone, 6 GB spare
BCLEAD=42  ->  43 steps  ->  ~64 GB     does not fit at all
```

and pruning earlier cannot help, because the peak IS the fetch.

`fetch_icon_global.sh` already has a per-file hook (`-x`) with `.done` markers and delete-on-success — the obvious wiring. **It is the wrong granularity, and the measurement is unambiguous.** Remapping one global file onto the San Juan 2 km grid gives a 505 KB netCDF of which only 65 KB is data; the rest is the LAM's coordinate and cell-bounds arrays, written once per FILE. Compression takes 942 KB to 505 KB and no further, because those arrays are what remains. Concatenate twelve fields and remap once and the whole file is **still ~507 KB**:

```
per-file hook   26,750 x 505 KB  ->  ~13.5 GB
per-step        43 x ~26 MB      ->   ~1.1 GB
```

The overhead is paid per file either way; the only question is how many files there are.

So `scripts/stream_global.sh` chunks by STEP: fetch a chunk, consume it into IC/BC, delete it, fetch the next. It needed almost no new code, because **preprocess_cdo.sh was already idempotent per step** — `build_bc_one` returns early when its latbc file exists. Two things were missing and are now in: `-F` **(a FIRST step)** on the fetcher, so a chunk does not refetch what was consumed and deleted, and an **IC cache guard**, so the IC is built once like the weights and the SST already were. Step 000 is never pruned: it holds the soil state and the time-invariant files, and `collect_step` reads those for every step.

Proven 27 Oct on that day's 06z, `BCLEAD=3 CHUNK=2`: chunk 0-1 built the IC and BCs and pruned 622 files; chunk 2-3 reported `weights cached / SST(LAM) cached / IC cached`, built only the new BCs and pruned 1,244 more. Peak ~3 GB where the whole cycle would have been 6.

Opt-in, via `GLOBAL_STREAM=1`**.** Default off, because the unstreamed path is the one proven end to end, and the switch is how the streamed one earns the same standing. One cosmetic wrinkle to know: the fetcher's closing `done: N/M files present` counts every `.grib2` in the directory, so with chunking it includes step 000's retained files and can read as more than the chunk's own total.

6. Implementation plan

Ordered so that nothing expensive is built before the thing that gates it, and nothing touches a racing morning.

Phase 0 — decide (done). ICON global, streamed, levels 45-120, 12z cycle. Everything below assumes it.

Phase 1 — prove the names and the grid, offline. No racing risk; the box is idle out of season.

1. Stage `icon_grid_0026_R03B07_G.nc` (937 MB) and check DWD's CDO library for **prebuilt weights** before generating our own.
2. **Pin CDO at 2.1.1.** 2.2.0 is broken on DWD global ICON GRIB2 (eccodes padding); 2.2.1 fixes it.
3. Prove `setgrid` + `gencon` on a single global file, following `preprocess_nest.sh` (which already does unstructured->unstructured), not `preprocess_cdo.sh`. Watch for "*Reference to source grid not found*" -> `selgrid` then `setgrid` then `gennn`.
4. Run `smoketest_tke.sh` against the **new variable names** (`FR_ICE`, `SNOAG`, and the snow set). An unknown output name aborts ICON at startup.

Phase 2 — the fetch. `fetch_icon_global.sh`, modelled on `fetch_icon_eu.sh`:

1. Level filter 45-120; steps 0-24; the extended variable list from 5e.
2. **Stream**: download -> bunzip2 -> remap -> append -> delete, so peak disk is a few files.
3. Keep the partial-`.bz2` deletion (a `wget -nc` that finds a corrupt file can never heal it).
4. Set `EXPECT_PER_STEP` ~775 and confirm it against the first real fetch's observed count.

Phase 3 — preprocessing.

1. Teach `preprocess_cdo.sh` the unstructured source (`-setgrid`), or branch to a global variant. Its header says "regular-lat-lon" — that assumption is the thing being changed.
2. **Never** `remapbil` **/** `remapbic` **on an ICON source.** Delete the "remapbil for winds if preferred" comment on line 46; it is an invitation to a failure that cannot happen on the Med.
3. Weights cached **per source->target pair** (riviera driving two nests already collided on a shared file).
4. Add `FR_ICE:fr_seaice` and `SNOAG:snow_age`; shrink `ZERO_SFC_2D` to the three that remain.

Phase 4 — grids. Only now submit the two Zonda requests (§2). They were deliberately held: a grid with no driving data runs nothing.

1. On arrival, place the `.nc`, then **dump** `z_mc` **and confirm** the level set.

Phase 5 — SST.

1. Test the premise first (§5b): if the Salish Sea is near-isothermal, 5.5 km is ample and the product question is closed cheaply.
2. Confirm the dataset-id with `copernicusmarine describe` before wiring. Keep `setmisstonn`.

Phase 6 — first run and integration.

1. Dry run on a past cycle with `REGATTA_ROOT` redirected, exactly as `smoketest_tke.sh` does.
2. `.env`: `DOMAINS`, `NESTS="san_juan_islands_2km:san_juan_islands_1km:18:4"`, `EVENT_VENUE`.
3. The app venues are already registered and tested; points stop greying out when the grid publishes.

Phase 7 — validate. Against HRRR (free, 3 km, already in SSA) and against the observations layer. If HRRR is as good between the islands, Phase 1-6 was the wrong investment and that is worth knowing.

6b. Checklist

- [x] Domains created, namelists verified byte-identical to Cowes
- [x] Zonda requests written and submitted, CORINE disabled with the ExtPar source checked
- [x] App venues registered on both resolutions, test pinned
- [x] **Driving data decided** (§6 phase 0): ICON global, streamed, levels 45-120, 12z
- [x] `fetch_icon_global.sh` written and dry-run (§5g); preprocessing source-aware (§5h); both roads serialised on one box (§5j); icosahedral chain proved (§5k)
- [x] App reads both status chains (`status.json` + `status_usa.json`), test pinned
- [x] SST resolution question answered by measurement (§5b): ~5.5 km is ample, choose on sovereignty
- [x] Zonda `.nc` placed into `grid/` and `extpar/`, verified against the working domain (27 Oct)
- [x] Pacific SST product chosen, measured and wired into `fetch_cmems_sst.sh` (27 Oct)
- [ ] `NESTS` / `DOMAINS` / `EVENT_VENUE` set in `.env` when the event approaches
- [ ] Confirm `z_mc` after the FIRST RUN (not after grid generation — it is not in the Zonda output)
- [ ] Dry run on a past cycle with `REGATTA_ROOT` redirected